

DEEP XSS PAYLOAD NOTES & METHODOLOGY GUIDE

ADVANCED CONTEXT-BASED BREAKTHROUGHS & VULNERABILITY RESEARCH LABS

CORE GOLDEN FORMULA: CONTEXT → ESCAPE → EXECUTE

1. EXECUTIVE CONCEPT BREAKDOWN

Cross-Site Scripting (XSS) occurs when an application receives untrusted data and routes it directly to a browser execution layer without contextual filtering or structural encoding. This transforms user inputs into runtime instructions, breaking the boundaries of the browser's *Same-Origin Policy (SOP)*.

Primary Strategic Objectives:

- **Session Hijacking & Cookie Theft:** Reading non-HttpOnly tokens via `document.cookie` to completely compromise state.
- **Credential Harvesting / Phishing:** Overlaying unauthenticated forms over current DOM matrices to grab plain-text fields.
- **Keylogging & Browser Exploitation:** Attaching remote scripts to capture live inputs or hooking browsers via command frame software.

2. ADVANCED VULNERABILITY CLASSIFICATIONS

- **Reflected XSS (Type-II):** The malicious payload arrives via an incoming HTTP packet request parameter (GET/POST) and maps directly to the immediate, single instance response stream without storage state.
- **Stored XSS (Persistent / Type-I):** The input vector commits fully to persistent memory structures (Database, Cloud Storage, File System logs). Every subsequent client retrieval fetches and evaluates the script natively.
- **DOM-Based XSS (Type-III):** The server payload output template never changes. Client-side execution occurs purely during client-side JavaScript execution, parsing runtime variables from local environment sources.
- **Stored DOM XSS:** A hybrid flow where persistent structures store the malformed payload strings, which are safely delivered via server-side formats, but execute when parsed inside dangerous browser execution points.

3. THE TAXONOMY OF EXPLOITATION CONTEXTS

A payload can never be written blindly; it relies entirely on the target parsing scope:

1. **HTML Body Context:** `<div>USER_INPUT</div>` — Solved by dropping valid alternative tags.
2. **HTML Attribute Context:** `<input value="USER_INPUT">` — Solved by terminating quotes and adding event handlers.
3. **JavaScript Inline Context:** `var label = 'USER_INPUT';` — Solved by closing internal literal boundaries.

4. **Textarea / Title / Select Container Context:** `<textarea>USER_INPUT</textarea>` — Requires explicit closure tags (`</textarea>`) first; otherwise, strings are seen as raw literal options.

4. DEEP VECTOR ENGINE ANALYSIS & PAYLOADS

`<script>alert(1)</script>`

Mechanism: Native execution tag structure.

Prerequisites: Raw HTML context; zero structural sanitization filters; absence of inline strict CSP barriers.

Failure Mode: Blocked when angle brackets are HTML-encoded to `<` and `>` .

`<img src=x onerror=alert(1)>`

Mechanism: Instant error fallback loop. `src=x` specifies an invalid element path, forcing the native browser asset worker to instantly shift to the `onerror` structural exception handler.

Advantages: Short, bypasses specific text filters checking for script blocks, handles execution immediately.

`<svg onload=alert(1)>`

Mechanism: Scalable Vector Graphics initialization. Instantiates an execution context immediately upon DOM element composition.

Strategic Target: Highly reliable when injecting strings into client-side `innerHTML` mutation sinks.

`" autofocus onfocus=alert(1) x="`

Mechanism: Attribute escape routine within `<input value="INPUT">` . The initial quote terminates the active attribute context. `autofocus` forces selection without user input, running the attached `onfocus` handler. The final trailing sequence cleans up broken attributes.

`';alert(1);// OR ";alert(1);//`

Mechanism: JavaScript String Breakout inside `var data = 'INPUT';` . The initial quote terminates the string boundary, the semicolon ends the declaration statement, your malicious command runs, and the double slash comments out trailing code lines.

`{{constructor.constructor('alert(1)')}()}}`

Mechanism: AngularJS Client Expression Injection. Used when standard HTML inputs map inside templates using double braces `{{1+1}}` . It accesses the JavaScript Function constructor scope to run code without direct script tags.

`"-alert(1)}//`

Mechanism: JSON Escape-Sequence Manipulation inside `eval(response)` . When software quotes are escaped by a backslash but the input doesn't clean backslashes, supplying a backslash cancels the application's escaping mechanism.

5. PARSER MANIPULATION & CONTEXT CLOSURE TARGETS

Browsers prioritize reconstructing broken layouts. Malformed structures like `<><img src=x onerror=alert(1)>` take advantage of parser error recovery to force the browser to exit a broken state and read the inner image vector correctly.

When input lands inside specialized document nodes, execution is paused unless you close out the parent container structure explicitly:

```
</textarea><svg onload=alert(1)>
</title><svg onload=alert(1)>
</select><svg onload=alert(1)>
</script><svg onload=alert(1)>
```

6. DOM ARCHITECTURE: SOURCES & SINKS MATRIX

DOM-based exploitation relies entirely on tracing untrusted input pathways (Sources) into dangerous compilation handlers (Sinks):

VULNERABLE ENTRY POINTS (SOURCES)	DANGEROUS EXECUTION AREAS (SINKS)	EXPLOITATION MECHANICS
<code>location.search</code> <code>location.hash</code>	<code>eval()</code> <code>Function()</code>	Directly evaluates the input string as executable JavaScript code inside the engine.
<code>document.URL</code> <code>document.referrer</code>	<code>element.innerHTML</code> <code>element.outerHTML</code>	Parses input strings directly into HTML elements, allowing tags like <code><svg></code> to run.
<code>window.name</code>	<code>document.write()</code>	Writes raw text directly to the document generation stream during loading.

7. AUTOMATION EVENT HANDLERS

- **Immediate / Auto-Triggered:** `onload`, `onerror`, `autofocus` + `onfocus`. Essential vectors when verifying bugs against automated headless verification bots.
- **User Interactive:** `onclick`, `onmouseover`, `onmouseenter`. Requires active user movement or interaction with the rendered container bounds.

8. STRATEGIC FAILURE ANALYSIS: WHY PAYLOADS FAIL

- **Structural HTML Encoding:** System maps characters like `<` to entities. *Remediation check:* See if quotes remain untouched to pivot to an Attribute Injection vector instead.
- **Mismatched Execution Bounds:** Injecting double-quote payloads into single-quoted parameters (e.g. `value='INPUT'`). The string remains bounded. Always inspect raw reflection attributes to use the matching quote type.
- **Content Security Policy (CSP):** Headers prevent inline code evaluation. *Remediation check:* Look for missing nonces or try script source reference overrides.

9. PROFESSIONAL BUG HUNTER WORKFLOW

Follow these operational phases when mapping out an asset scope:

```
[PHASE 1: RECON]      --> Inject safe structural identifier string (e.g., "zXyW77").
[PHASE 2: ANALYZE]    --> Inspect reflection point via DevTools. Identify precise context
node.
[PHASE 3: PROFILE]    --> Test character filtering constraints ('', "< > \, ;, /).
[PHASE 4: STRATEGY]   --> Apply Golden Rule: Escape current context context boundary.
[PHASE 5: INSTRUMENT] --> Deploy corresponding vector payload (e.g., Auto-Triggered Event).
[PHASE 6: CONFIRM]    --> Verify script execution inside the active DOM runtime environment.
```